Mock National Championship

12 problems · 5 hours · 1 computer per team · then a real post-mortem

Learning goals

- Run a full national-championship simulation under real conditions
- Apply Lectures 1–9 **without being told which technique to use**
- Practise triage on a set where the easy problems are **not** in order
- Produce a structured **post-mortem** that turns mistakes into action items
- Leave with an updated team notebook and a personal training plan

Format and rules

- Teams of 3, **one computer**, 5 hours, printed team notebook (≤ 25 pages)
- ICPC scoring: problems solved, then penalty = Σ (solve minute + 20 × rejected runs)
- Scoreboard freeze in the last hour
- C++17, `-O2`; stack limit = default 8 MB (beware deep recursion!)
- Statements are on the handout; this deck's problem slides are for the **debrief**

Session plan

Time	Activity
0:00 – 0:10	Setup, notebook check, template typed
0:10 – 5:10	Contest
5:10 – 5:40	Break; teams fill in the **raw timeline**
5:40 – 7:00	Debrief: problem-by-problem solutions
Next session	Team post-mortem presented (10 min/team), upsolving targets set

The problem set

	Problem	Area	Est. rating
A	Lanterns	greedy	1300
B	Recurring Revenue	matrix power, many queries	2100
C	Toll Bridges	bridge tree + LCA	2000
D	Distance Band	centroid decomposition	2300
E	Echo	suffix array / SAM	1900
F	Corridor	hull + rotating calipers	1900
G	Overtime Assignment	min-cost flow, convex costs	2300
H	Twin Stations	binary search + 2-SAT	2200
I	Xor Treasure Map	cycle space, xor basis	2300

A — Lanterns (1300)

Task. Street [0, L]; lantern i lights $[p_i - r_i, p_i + r_i]$. Minimum number of lanterns covering all of [0, L], or −1. $n \le 2 \cdot 10^5$.

Idea. Classic interval cover: sort by left end; `cur = 0`; repeatedly take the largest right end among intervals with left ≤ cur. If it does not exceed cur → −1.

Trap. Loop while `cur < L` (not ≤), 64-bit sums, unsorted input.

O(n log n).

E — Echo (1900)

Task. String s ($n \le 2 \cdot 10^5$) and k. Longest substring occurring ≥ k times (overlaps allowed).

Idea. SA + LCP: answer = max over windows of k − 1 consecutive lcp values of their minimum (sliding-window min). k = 1 ⇒ n.

Alternatives: SAM — max `len(v)` with `occ(v) ≥ k`; or binary search + hashing $O(n \log^2 n)$.

$O(n \log n)$.

F — Corridor (1900)

Task. $n \le 2 \cdot 10^5$ integer points. Minimum width of an infinite straight strip containing them all.

Idea. Width of the convex hull. Optimal strip has one side flush with a hull edge ⇒ rotating calipers: for each edge, the farthest vertex; width = min area/edge length.

Trap. All collinear ⇒ 0; hull with collinear points stalls the caliper; overflow in cross products ($\le 8 \cdot 10^{18}$ is fine, squares of it are not).

O(n log n).

C — Toll Bridges (2000)

Task. Connected multigraph, n, m, q ≤ $3 \cdot 10^5$. Query (u, v): how many edges, if removed alone, disconnect u from v?

Idea. Exactly the bridges on any u–v path. Contract 2-edge-connected components ⇒
bridge tree. Answer = distance between comp(u) and comp(v) in that tree (LCA).

Trap. Multi-edges (skip parent **edge id**), recursion depth $3 \cdot 10^5$ ⇒ iterative DFS.

O((n + m) + q log n). Reference code in the notes.

K — Lighthouse (2000)

Task. $n \le 2 \cdot 10^5$ ships with values at integer points ≠ origin. A beam is a closed sector of angle α (integer degrees) with apex at the origin. Max total value inside one beam position.

Idea. Sort ships by angle; duplicate with +2π; two pointers keep the window $[\theta_i, \theta_i + \alpha]$; track window sum. Ships in the same direction move together.

Trap. α ≥ 180° still works with the doubled array; never let the window exceed n ships.

O(n log n).

B — Recurring Revenue (2100)

Task. $a_i = \sum_{j=1..k} c_j\, a_{i-j} + d\cdot i + e \pmod{10^9+7}$, $k \le 20$. $q \le 2000$ queries for a_n, $n \le 10^{18}$.

Idea. State $(a_{i-1}..a_{i-k}, i, 1)$, size $k + 2$. Precompute T^(2^j) for $j < 60$.
Each query multiplies a **vector** by ≤ 60 matrices: $O(k^2 \log n)$.

Trap. Per-query matrix powering is $2000\cdot 60\cdot 22^3 \approx 1.3\cdot 10^9$ — too slow.

H — Twin Stations (2200)

Task. n ≤ 1000 stations; station i goes at position a_i or b_i. Maximize the minimum distance between any two stations.

Idea. Binary search D. Variable x_i = "station i at a_i". For every pair of choices at distance < D add clause $\neg(\text{choice}_1 \wedge \text{choice}_2)$. Feasible ⇔ 2-SAT satisfiable.

$O(n^2 \log C)$. About $4 \cdot 10^6$ implication edges per check — reuse buffers.

J — Dice Tower (2200)

Task. n dice; die i has faces with values in $[0, 10^5]$. Total faces $\le 2 \cdot 10^5$ and $S = \Sigma$ (max face) $\le 2 \cdot 10^5$. For each total $t \in [0, S]$, count outcomes mod 998244353.

Idea. Product of n polynomials. Multiply in a balanced way: divide & conquer or always merge the two **smallest** (priority queue). Every coefficient participates in $O(\log n)$ NTTs $\Rightarrow O(S \log^2 S)$.

Trap. Multiplying left to right is $O(n \cdot S \log S)$.

D — Distance Band (2300)

Task. Tree, $n \le 2 \cdot 10^5$. Count unordered pairs with $L \le \text{dist}(u, v) \le R$.

Idea. $f(X)$ = #pairs with dist $\le X$; answer $f(R) - f(L - 1)$.
Centroid decomposition: at centroid c, depth counts of the whole piece; pairs with
$d_1 + d_2 \le X$ via prefix sums; subtract pairs inside the same child subtree.

$O(n \log n)$ (depth arrays are bounded by piece sizes). Variant with FFT gives counts for all distances.

G — Overtime Assignment (2300)

Task. n, m ≤ 100. Worker i can do task j for c_{ij} (or not at all). Each task done once. A worker doing t tasks pays extra $o_i \cdot t(t-1)/2$. Minimize total cost.

Idea. Min-cost flow. Source → worker i: m unit edges with costs 0, o_i, $2o_i$, … (convex, so they are used in order). Worker → task (cap 1, cost c_{ij}). Task → sink. Need flow m.

$O(m \cdot (V + E) \log V)$ with Dijkstra + potentials.

I — Xor Treasure Map (2300)

Task. Connected graph, weights < 2^{60}, n, m, q ≤ $2 \cdot 10^5$. Query (u, v): max xor over all walks u → v (edges reusable).

Idea. Any walk = tree path xor $\oplus$ subset of cycle values. DFS tree with d[v]; each non-tree edge gives cycle d[a] $\oplus$ d[b] $\oplus$ w → insert into one global basis. Answer = maxXor(d[u] $\oplus$ d[v]).

O((m + q) · 60). Reference code in the notes.

L — Warehouse Districts (2400)

Task. Positive $a_i \le 10^4$, $n \le 10^5$. Split into exactly $k \le 100$ contiguous parts minimizing Σ (part sum)^2.

Idea. dp[j][i] = S_i^2 + min_p (dp[j−1][p] + S_p^2 − 2 S_i S_p). Lines with slope $-2S_p$ (decreasing), queries S_i increasing ⇒ monotone deque CHT, O(nk).
Alternative: D&C optimization O(kn log n).

Trap. Intersection test products reach ~10^{27} ⇒ `__int128` or `long double`.

Expected difficulty curve



Gold-medal pace: 9–10 problems. Advance-to-finals pace: 7–8. Solid: 5–6.

The set is **not** sorted: E, F, C, K are as easy as they look — did you find them in the first hour?

Post-mortem: process

1. **Raw timeline** (immediately): every read, submit, verdict, who was on the keyboard
2. **Per-problem table**: status, first-read time, idea time, AC time, wrong runs, root cause
3. **Root-cause categories**: reading · idea · implementation · edge case · strategy/time
4. **Counterfactual**: "with perfect play on what we *knew*, what was our score?"
5. **Three action items** max, each concrete and testable
6. **Notebook diff**: what code was missing or wrong?

Post-mortem template (per problem)

Field	Example
Problem / status	H — AC at 3:41, 2 WA
First read by / at	Alice, 0:12
Idea found at	2:50 (binary search + 2-SAT)
Why so late?	Stuck on greedy; nobody re-read after 1:00
WA causes	inverted Tarjan assignment; clause for a = b
Fix / action	add 2-SAT self-test to notebook; re-read rule at 1:30

Metrics worth tracking across mocks

- Time to first AC; time to 3 ACs
- Idle-keyboard minutes
- Wrong submissions per AC (target < 0.5)
- Problems where the idea was known but not coded in time
- "Read but never attempted" problems that turned out easy
- Upsolve completion within one week (target: every problem ≤ 2400)

Common contest-level mistakes

- Reading in letter order and missing easy E/F/K
- Two people silently working on the same problem
- Coding before writing down the idea and the complexity
- No stress test after the second WA
- Leaving the computer idle while someone thinks
- Freeze-hour panic: submitting unverified "fixes"

Practice after the mock

- Upsolve every mock problem; reference ideas in the notes
- Real sets to simulate next: ICPC North America Championship sets (Kattis), NERC finals (Codeforces Gym / contest 1912), NWERC / SWERC on Codeforces Gym
- Per-topic follow-ups: CF 1000E (C), CF 845G (I), CF 311B and 321E (L), CF 161D and 293E (D), POJ 2296 (H), CF 1251F (J), CF 1284E (K), CF 277E (G), CF 1182E (B)

Summary

- A mock is only as valuable as its **post-mortem**
- Map each miss to a category; fix the category, not the instance
- Update the notebook the same week
- Next: World Finals track — mindset, polynomials, matching & LP, advanced trees